

Etapa 1 – Analisador Léxico em Flex

Livia Maria Almeida Silva
Universidade Federal de Lavras – UFLA
Departamento de Ciência da Computação
Lavras, Minas Gerais, Brasil

1 Introdução

Este relatório apresenta a primeira etapa do trabalho de Compiladores, cujo objetivo é desenvolver o analisador léxico, em *Flex*, de uma linguagem simplificada. O analisador reconhece as classes léxicas exigidas, informa *token*, lexema, linha e coluna, mantém uma tabela de símbolos para identificadores e reporta entradas inválidas com sua posição.

2 Decisões de Projeto

Palavras reservadas e tipos são reconhecidos por regras próprias e não são inseridos na tabela de símbolos. Identificadores seguem $[A-Za-z][A-Za-z][0-9]^*$; a tabela armazena até 100000 identificadores distintos, com no máximo 100 caracteres, usando a posição no vetor como atributo de ID. Números inteiros e decimais seguem $[0-9]^+([0-9]^+)?$ e compartilham o *token* NUMERO. Operadores relacionais e tipos são agrupados no mesmo *token*. Os demais operadores possuem *tokens* próprios. Valores negativos são decompostos em MENOS seguido de NUMERO, deixando a interpretação do sinal para etapas posteriores. Comentários de uma ou várias linhas são descartados; nos multilinha, a coluna é atualizada caractere a caractere. Também são detectados caractere inválido, comentário multilinha não fechado e estouro dos limites da tabela.

3 Dificuldades Encontradas

As principais dificuldades foram decidir o tratamento de números negativos, manter corretamente linha/coluna após comentários multilinha e fazer tratamento de erro para comentários multilinha não fechados.

4 Definição da Linguagem

A linguagem possui sintaxe simplificada inspirada em C. São suportados os tipos `int` e `float`, identificadores iniciados por letra e literais numéricos inteiros ou decimais. As estruturas da linguagem incluem `if`, `else`, `while`, `read` e `print`.

As expressões podem utilizar operadores aritméticos (+ - * / %), relacionais (< <= > >= == <>) e lógicos (&& || !). Blocos são delimitados por chaves e a linguagem também utiliza ; , e parênteses como pontuação. Comentários podem ser escritos com // ou /* ... */.

Exemplo de sintaxe:

```
int valor;
if (valor >= 10) {
    print(valor);
}
```

5 Diagramas de Transição

A Fig. 1 apresenta os DFAs usados como referência para as classes NUMERO (superior) e ID (inferior).

Mauricio Vicente Sandim
Universidade Federal de Lavras – UFLA
Departamento de Ciência da Computação
Lavras, Minas Gerais, Brasil

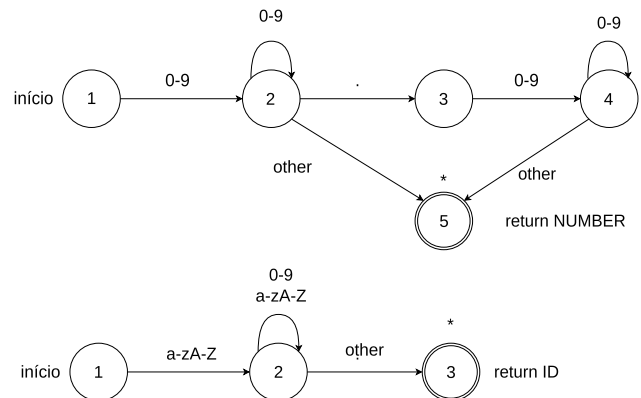


Figure 1: Diagramas de transição para números e identificadores.

6 Testes e Resultados

O teste abaixo cobre todas as classes léxicas, comentários e uma entrada inválida. A saída apresentada é a produzida pelo analisador.

Entrada:

```
int float if else while print read
id id1 0.546 13.5 -26
+ * / % <= > == <> && || ! ; , ( ) { }
// comentario
/* comentario
comentario 2*/
@
```

Saída:

```
Linha: 1; Coluna: 1; Lexema: int; Token: <TIPO, int>
Linha: 1; Coluna: 5; Lexema: float; Token: <TIPO, float>
Linha: 1; Coluna: 11; Lexema: if; Token: <IF>
Linha: 1; Coluna: 14; Lexema: else; Token: <ELSE>
Linha: 1; Coluna: 19; Lexema: while; Token: <WHILE>
Linha: 1; Coluna: 25; Lexema: print; Token: <PRINT>
Linha: 1; Coluna: 31; Lexema: read; Token: <READ>
Linha: 2; Coluna: 1; Lexema: id; Token: <ID, 0>
Linha: 2; Coluna: 4; Lexema: id1; Token: <ID, 1>
Linha: 2; Coluna: 8; Lexema: 0.546; Token: <NUMERO, 0.546>
Linha: 2; Coluna: 14; Lexema: 13.5; Token: <NUMERO, 13.5>
Linha: 2; Coluna: 19; Lexema: -; Token: <MENOS>
Linha: 2; Coluna: 20; Lexema: 26; Token: <NUMERO, 26>
Linha: 3; Coluna: 1; Lexema: +; Token: <MAIS>
Linha: 3; Coluna: 3; Lexema: *; Token: <MULT>
Linha: 3; Coluna: 5; Lexema: /; Token: <DIV>
Linha: 3; Coluna: 7; Lexema: %; Token: <MOD>
Linha: 3; Coluna: 9; Lexema: <=; Token: <RELOP, <= >
Linha: 3; Coluna: 12; Lexema: >; Token: <RELOP, > >
Linha: 3; Coluna: 15; Lexema: ==; Token: <RELOP, == >
Linha: 3; Coluna: 18; Lexema: <>; Token: <RELOP, < > >
Linha: 3; Coluna: 21; Lexema: <; Token: <RELOP, < >
Linha: 3; Coluna: 23; Lexema: >; Token: <RELOP, > >
Linha: 3; Coluna: 25; Lexema: &&; Token: <E>
Linha: 3; Coluna: 28; Lexema: ||; Token: <OU>
Linha: 3; Coluna: 31; Lexema: !; Token: <NAO>
Linha: 3; Coluna: 33; Lexema: ;; Token: <PONTO_VIRGULA>
Linha: 3; Coluna: 35; Lexema: ,; Token: <VIRGULA>
Linha: 3; Coluna: 37; Lexema: (; Token: <ABRE_PARENTESES>
Linha: 3; Coluna: 39; Lexema: ); Token: <FECHA_PARENTESES>
Linha: 3; Coluna: 41; Lexema: {; Token: <ABRE_CHAVES>
Linha: 3; Coluna: 43; Lexema: }; Token: <FECHA_CHAVES>
Erro léxico na linha 7 e coluna 1. Entrada -> "@"
```

===== TABELA DE SÍMBOLOS =====	
ID	LEXEMA
0	id
1	id1